CHAPTER 5 — SYSTEM DESIGN

5.1 Introduction

This chapter presents the complete system design of UniConnect, the peer-learning platform built exclusively for Philadelphia University students. The design covers the system architecture, database schema, entity-relationship mapping, class and service structure, user interface design, security design, and the points and rewards system. All design decisions are grounded in the functional and non-functional requirements established in Chapter 4, and align with the Agile/Scrum methodology described in Chapter 3.

The design follows three core architectural patterns: Layered Architecture for clear separation of concerns, Model-View-Controller (MVC) for the web application structure, and the Repository Pattern for data access abstraction. These patterns ensure the system remains maintainable, testable, and scalable as the user base grows.

5.2 System Architecture

5.2.1 Architectural Overview

UniConnect adopts a three-tier Layered Architecture consisting of the Presentation Layer, Business Logic Layer, and Data Access Layer. This clear separation ensures that changes in one layer do not directly affect others, directly supporting the maintainability requirement (NFR6). The three tiers communicate as follows:

1. Presentation Layer: The web browser renders HTML5/CSS3/JS pages served by ASP.NET Core MVC Razor Views.
2. Business Logic Layer: ASP.NET Core controllers and service classes handle authentication, Q&A, points, sessions, search, and moderation.
3. Data Access Layer: Repository classes using Entity Framework Core communicate with the SQL Server database through parameterized LINQ queries.

5.2.2 Technology Stack

The following technology stack was selected based on team expertise, university environment requirements, and the scalability needs of the platform:

Layer	Technology	Justification
Presentation / Frontend	`HTML5, CSS3, JavaScript`	Universal browser support; no framework dependency required for the prototype phase.
Application / Backend	`ASP.NET Core (C#)`	High performance, strongly typed; well suited to MVC web applications.
Data / Database	`Microsoft SQL Server`	Enterprise-grade RDBMS; integrates seamlessly with the .NET ecosystem via EF Core.
ORM	`Entity Framework Core`	Reduces boilerplate data-access code; supports code-first migrations and LINQ queries.

Layer	Technology	Justification
Authentication	`Cookie Authentication (ASP.NET Core Cookies)`	Session-based; HttpOnly encrypted cookie issued on login, validated on every request. Appropriate for server-rendered MVC applications (FR1).
Version Control	`Git / GitHub`	Team collaboration and branching strategy aligned with Scrum sprints.

Table 5.1: UniConnect Technology Stack

5.3 Database Design (ERD)

5.3.1 Overview

The UniConnect database is designed using a relational model hosted on Microsoft SQL Server and managed through Entity Framework Core code-first migrations. The schema consists of eleven tables that together satisfy all functional requirements FR1–FR11 defined in Chapter 4. Primary and foreign key constraints enforce referential integrity throughout the schema (NFR5).

The eleven tables are: Users, Posts, Answers, Categories, Tags, PostTags, Requests, PrivateSessions, Messages, Reports, and Notifications. Points are not stored in a separate table; they are maintained as an integer column (Points) in the Users table and updated by the application layer whenever a point-earning or point-spending event occurs.

5.3.2 Entity Descriptions

Table 5.2 describes all eleven database entities, their primary keys, and their roles within the system:

Table Name	Primary Key	Description
Users	`UserID (INT, PK, IDENTITY)`	All registered student and admin accounts. Includes Points balance as an integer column, plus security fields (FailedLoginAttempts, AccountLockedUntil, PasswordResetToken).
Posts	`PostID (INT, PK, IDENTITY)`	Academic questions submitted by students. Linked to Users (author) and Categories (faculty/course). Supports Upvotes and ViewsCount. Satisfies FR3, FR4.
Answers	`AnswerID (INT, PK, IDENTITY)`	Responses to Posts. Tracks IsAccepted (best answer flag) and Upvotes. Linked to Posts and Users. Satisfies FR3, FR4.
Categories	`CategoryID (INT, PK, IDENTITY)`	Faculty and course groupings used to organise and filter Posts (Name, Faculty columns). Satisfies FR3, FR6.
Tags	`TagID (INT, PK, IDENTITY)`	Keyword labels that can be attached to Posts. Linked through PostTags junction table. Satisfies FR3.
PostTags	`PostID + TagID (Composite PK)`	Junction table implementing the many-to-many relationship between Posts and Tags. Each row links one Post to one Tag. No extra columns.
Requests	`RequestID (INT, PK, IDENTITY)`	A student's request to open a private tutoring session with the owner of a specific Post. Status: Open / Accepted / Closed. Satisfies FR4, FR8.

Table Name	Primary Key	Description
PrivateSessions	`PrivateSessionID (INT, PK, IDENTITY)`	An active or completed private tutoring session created when a Request is accepted. References two Users: Student and Helper. Satisfies FR4, FR8.
Messages	`MessageID (INT, PK, IDENTITY)`	Individual chat messages exchanged inside a PrivateSession. Tracks IsRead and SentAt. Satisfies FR8.
Reports	`ReportID (INT, PK, IDENTITY)`	User-submitted reports flagging inappropriate Post content. Includes Reason and Status (Pending / Reviewed). Satisfies FR7.
Notifications	`NotificationID (INT, PK, IDENTITY)`	System notifications dispatched to users (new answers, session updates, points events, etc.). Tracks IsRead, Type, and ReferenceID. Satisfies FR11.

Table 5.2: UniConnect Database Entities (11 Tables)

5.3.3 ERD — Primary Keys and Foreign Keys

Table 5.3 provides the complete entity-relationship mapping showing every table's primary key and all foreign key references. This replaces the ERD diagram and presents the same structural information in tabular form as required.

Table	Primary Key (PK)	Foreign Keys (FK)	Key Relationships
Users	`UserID`	—	Root entity. Referenced by all tables. Points balance is a column here, not a separate table.
Posts	`PostID`	`UserID → Users` `CategoryID → Categories`	Each post belongs to one author and one category.
Answers	`AnswerID`	`PostID → Posts` `UserID → Users`	Each answer belongs to one post and one author.
Categories	`CategoryID`	—	Referenced by Posts. Represents faculty and course groupings.
Tags	`TagID`	—	Referenced by PostTags. Keyword labels for posts.
PostTags	`PostID + TagID (Composite PK)`	`PostID → Posts` `TagID → Tags`	Junction table for many-to-many between Posts and Tags. Both columns are FK and together form the PK.
Requests	`RequestID`	`OwnerID → Users` `PostID → Posts`	A session request targeting a specific post owner.
PrivateSessions	`PrivateSessionID`	`RequestID → Requests StudentID → Users HelperID → Users`	Accepted session. Two separate FK references to Users (student + helper). DeleteBehavior.NoAction on both.
Messages	`MessageID`	`SessionID → PrivateSessions` `SenderID → Users`	Individual message in a session. SenderID uses NoAction delete.

Table	Primary Key (PK)	Foreign Keys (FK)	Key Relationships
Reports	`ReportID`	`ReporterID → Users` `PostID → Posts`	Report submitted by a user against a post.
Notifications	`NotificationID`	`UserID → Users`	Notification dispatched to a specific user. NoAction on delete.

Table 5.3: ERD Table Specification — Primary Keys and Foreign Keys

Key structural notes:

- **Many-to-Many (Posts ↔ Tags):** Resolved through the PostTags junction table. PostID and TagID together form a composite primary key; each column is simultaneously a foreign key to its parent table.
- **Two-role relationship (PrivateSessions → Users):** PrivateSessions references Users twice — once as StudentID and once as HelperID. Both foreign keys use DeleteBehavior.NoAction to prevent cascade conflicts.
- **Soft Delete:** Eight tables (Users, Posts, Answers, Requests, PrivateSessions, Messages, Notifications, Reports) include an IsDeleted boolean column. A global EF Core query filter (HasQueryFilter) automatically excludes deleted rows from all queries.

5.4 Class Diagram

The UniConnect backend follows object-oriented design principles. Table 5.4 presents the complete class structure including all model attributes and service operations for each entity. The operations listed reflect all features described in FR1–FR11.

Class / Service	Attributes & Operations	Design Pattern / Notes
User	`UserID, UniversityID, Name, Username, Email, PasswordHash, Role, Points, ProfileImageUrl, CreatedAt, LastLoginAt, FailedLoginAttempts, AccountLockedUntil — Register(), Login(), Logout(), GetProfile(), UpdateProfile(), ResetPassword(), LockAccount()`	Repository Pattern — UserRepository abstracts all DB access. Observer Pattern — triggers Notification on login events.
Post	`PostID, UserID, CategoryID, Title, Content, ViewsCount, Upvotes, CreatedAt, IsDeleted — CreatePost(), EditPost(), DeletePost(), GetPost(), ListPosts(), FilterByCategory(), SearchPosts(), UpvotePost()`	Repository Pattern — PostRepository. Soft Delete via IsDeleted flag + EF query filter.
Answer	`AnswerID, PostID, UserID, Content, IsAccepted, Upvotes, CreatedAt, IsDeleted — CreateAnswer(), EditAnswer(), DeleteAnswer(),`	Repository Pattern — AnswerRepository. Observer: triggers Notification when answer is added or accepted.

Class / Service	Attributes & Operations	Design Pattern / Notes
	`MarkAsBestAnswer(), UpvoteAnswer()`	
Category	`CategoryID, Name, Faculty — GetCategories(), CreateCategory(), DeleteCategory()`	Admin-managed. Referenced by Post. No soft delete.
Tag	`TagID, Name — CreateTag(), GetTagsByPost(), AssignTagToPost(), RemoveTagFromPost()`	Many-to-many with Posts via PostTags junction.
Request	`RequestID, OwnerID, PostID, Description, Status, CreatedAt, IsDeleted — CreateRequest(), AcceptRequest(), DeclineRequest(), CloseRequest(), GetRequestsByPost()`	Status flow: Open → Accepted → Closed. Creating a Request costs the requester −5 points.
PrivateSession	`PrivateSessionID, RequestID, StudentID, HelperID, CreatedAt, IsActive, IsDeleted — StartSession(), EndSession(), GetSessionMessages(), RateSession()`	Created only when a Request is Accepted. Two FK refs to Users (NoAction cascade).
Message	`MessageID, SessionID, SenderID, MessageText, SentAt, IsRead, IsDeleted — SendMessage(), GetMessages(), MarkAsRead(), DeleteMessage()`	Belongs to PrivateSession. Observer: triggers Notification to recipient.
Report	`ReportID, ReporterID, PostID, Reason, Status, IsDeleted — SubmitReport(), ReviewReport(), DismissReport()`	Status: Pending → Reviewed. Admin-only ReviewReport() and DismissReport().
Notification	`NotificationID, UserID, Type, ReferenceID, IsRead, CreatedAt, IsDeleted — SendNotification(), MarkAsRead(), GetUnreadCount(), GetAllNotifications()`	Observer Pattern — NotificationService listens for Answer, Upvote, Session, Report events and dispatches notifications automatically.

Table 5.4: UniConnect Class Structure — Attributes and Operations

Table 5.5 summarises the four design patterns applied across the system:

Pattern	Where Applied	Purpose
Repository	All data-access classes	Abstracts EF Core queries from business logic; enables unit testing and future database migration without changing service code.
Observer	NotificationService	Automatically notifies users when events occur: new answers, upvotes, session requests, or report updates.

Pattern	Where Applied	Purpose
MVC	ASP.NET Core Controllers	Separates presentation (Views), business logic (Controllers/Services), and data models cleanly across all features.
Soft Delete	Users, Posts, Answers, Requests, PrivateSessions, Messages, Notifications, Reports	IsDeleted flag + EF Core global query filter (HasQueryFilter) ensures deleted records are excluded from all queries without physical removal.

Table 5.5: Design Patterns Applied in UniConnect

5.5 User Interface Design

The UniConnect interface is accessible via Figma at: https://www.figma.com/design/VriK6QafriBYw9CPGZvjZ0/1

5.5.1 Design Principles

The UniConnect interface was designed following established UX principles to ensure ease of use for university students across all faculties (NFR4 — Usability). The following five principles guided all design decisions:

- **Consistency:** All 25 pages share the same navigation structure, color palette (Primary Indigo #3D52A0, Amber #F59E0B), typography (Plus Jakarta Sans), and component library.
- **Visual Hierarchy:** Important actions use the primary indigo color. Secondary actions use ghost buttons. Destructive actions use red.
- **Feedback:** Every action provides immediate visual feedback — toast notifications, loading indicators, form validation messages, and character counters.
- **Accessibility:** Minimum contrast ratios maintained. Form inputs have clear labels and error messages. Navigation functions with keyboard.
- **Responsive Design:** All pages are fully responsive. The sidebar collapses into a hamburger drawer on screens below 768px width.

5.5.2 Page Inventory (25 Pages)

#	File Name	Page Name	Requirement
1	`index.html`	Landing Page (visitors)	Public marketing page
2	`login.html`	Sign In	FR1
3	`register.html`	Register	FR1
4	`register-success.html`	Email Verification Sent	FR1
5	`forgot-password.html`	Forgot Password	FR1
6	`reset-password.html`	Reset Password	FR1

#	File Name	Page Name	Requirement
7	`dashboard.html`	Home Feed	FR3, FR6
8	`create-post-step1.html`	Ask — Step 1: Search First	FR3, FR6
9	`create-post-step2.html`	Ask — Step 2: Write Question	FR3
10	`create-post-step3.html`	Ask — Step 3: Preview & Post	FR3
11	`create-post-step4.html`	Ask — Step 4: Success	FR3, FR5
12	`single-post.html`	Question Detail + Answers	FR3, FR4
13	`profile.html`	My Profile	FR2
14	`user-profile.html`	Other User Profile	FR2
15	`sessions.html`	Private Sessions + Chat	FR4, FR8
16	`leaderboard.html`	Leaderboard	FR5
17	`notifications.html`	Notifications	FR11
18	`points.html`	Points & Campus Rewards	FR5
19	`settings.html`	Settings	FR2
20	`search-results.html`	Full Search Results	FR6
21	`report-content.html`	Report Content	FR7
22	`admin.html`	Admin Dashboard	FR9
23	`admin-user-detail.html`	Admin User Detail	FR9
24	`404.html`	Page Not Found	NFR4
25	`create-post.html`	Ask a Question (combined)	FR3

Table 5.6: UniConnect 25-Page Inventory

5.6 Security Design

Security is enforced at every layer of the UniConnect architecture, aligned with NFR3. The following measures are implemented in the actual codebase (Program.cs and LoginController.cs) to protect user data and prevent unauthorized access:

Security Area	Measure	Implementation
Authentication	Cookie Authentication	ASP.NET Core Cookie middleware issues an encrypted, HttpOnly session cookie on login. Cookie expires after 24 hours with SlidingExpiration enabled. No token stored in JavaScript-accessible storage.
Passwords	BCrypt Hashing	BCrypt.Net.BCrypt.HashPassword() before storage. BCrypt.Verify() on login. Salt is embedded in each hash, so identical passwords produce different stored values.
Brute-Force Protection	Account Lockout	5 consecutive failed login attempts locks the account for 15 minutes (AccountLockedUntil column). Implemented directly in LoginController.
Authorization	Role-Based Access Control	Two roles: Student and Admin. Role claim embedded in the cookie at login. [Authorize(Roles="Admin")] attribute protects admin endpoints. Students cannot access admin pages.
SQL Injection	Parameterized Queries	All database access uses Entity Framework Core LINQ queries. EF Core generates parameterized SQL; raw SQL is not used anywhere in the repository.
CSRF Protection	Anti-Forgery Tokens	[ValidateAntiForgeryToken] applied on all POST actions in LoginController. ASP.NET Core generates and validates __RequestVerificationToken automatically.
University Access Lock	Email Domain Validation	Registration rejected if email does not end in @philadelphia.edu.jo. Checked in LoginController.Register_Page() before any DB write.
Transport Security	HTTPS / HSTS	app.UseHttpsRedirection() and app.UseHsts() configured in Program.cs. All HTTP requests redirected to HTTPS.

Table 5.7: UniConnect Security Measures

5.7 Points and Rewards System Design

The Points and Rewards System (FR5) is a central feature of UniConnect that incentivises academic contribution. Points are stored as an integer column in the Users table and are updated atomically by the application layer on every qualifying event. When a student registers, they immediately receive +50 welcome points (implemented in LoginController.Register_Page()).

When a student redeems a campus venue reward, the system: (1) verifies sufficient balance, (2) deducts the points atomically, (3) generates a unique QR code, and (4) sends it to the student's university email.

5.7.1 Points Transaction Rules

Action	Points	Trigger
Register new account	**+50**	One-time welcome bonus. Applied immediately on successful registration.
Post an answer	**+5**	Answer submitted to any question.
Receive an upvote on answer	**+10**	Per upvote received on a submitted answer.

Action	Points	Trigger
Answer marked as Best Answer	**+15**	Question owner selects best answer.
Accept a tutoring session	**+20**	Session accepted and completed.
Receive 5-star session rating	**+15**	Tutee rates the session 5 out of 5.
Daily login	**+2**	First login of each calendar day.
Post a question	**−10**	Question submitted to the platform.
Request a tutoring session	**−5**	Session request sent to a tutor.

Table 5.8: Points Transaction Rules

5.7.2 Campus Venue Redemptions

Venue	Discount	Cost (pts)	Location
University Cafeteria	10% off any meal	200	Main Building, G/F
University Library	5 JD credit	400	Library Building, Level 2
Campus Bookshop	15% off purchases	350	Student Centre, L1
Print & Copy Centre	20 free A4 pages	150	IT Building, Room 102
Student Canteen	Free hot drink	180	Engineering Building
Sports & Gym Centre	1 week free access	500	Sports Complex
Computer Lab Booking	2 hours free access	100	IT Building, Lab 3
Extended Library Access	Stay until midnight (1 month)	300	Library Building
Verified Tutor Badge	Platform badge + privileges	1,000	Profile badge (digital)

Table 5.9: Campus Partner Venues and Redemption Costs

5.8 Summary

This chapter presented the complete system design of UniConnect across six key areas:

4. **System Architecture:** A three-tier Layered Architecture with ASP.NET Core MVC backend, SQL Server database managed via Entity Framework Core, and HTML/CSS/JS frontend using MVC and Repository patterns.
5. **Database Design:** Eleven relational tables covering users, posts, answers, categories, tags (with PostTags junction), requests, private sessions, messages, reports, and notifications. Full PK/FK mapping provided in Table 5.3.

6. **Class Diagram:** Ten domain classes with full attribute and operation lists. Four design patterns applied: Repository, Observer, MVC, and Soft Delete.
7. **User Interface Design:** 25 self-contained HTML pages organised into five functional groups, built on a consistent design system with full responsive layout.
8. **Security Design:** Multi-layer security including Cookie Authentication, BCrypt password hashing, account lockout after 5 failed attempts, role-based access control, parameterized queries, CSRF protection, and university email domain restriction.
9. **Points System:** A gamification engine with seven earn actions, two spend actions, and nine campus venue redemption options. Points stored as a column in Users.

The design decisions in this chapter directly address all functional requirements (FR1–FR11) and non-functional requirements (NFR1–NFR7) established in Chapter 4. The backend implementation using ASP.NET Core and SQL Server is scheduled for completion by April 25, 2026.